From Operation Logs to an Automation Proposal

Final report · Repository: https://github.com/insaneado/iby

Summary

Dataset B contains **173 minutes of back-office work: 664 executions across 15 sessions and 4 operators**, all on 2026-07-01. Recovering that required solving the problem the raw logs pose — they record keystrokes and clicks, but nothing that says which unit of work is in progress.

The three findings that drove everything else:

1. **The case identifier is recoverable from the screen.** 97.8% of ground-truth case IDs appear in captured screen text. This turns Step 1 from blind change-point detection into case-identity reconstruction — the only method that can separate two consecutive executions of the *same* process.
2. **Each unit of work is bracketed by two observable clicks.** Selecting a record opens it (1,780 clicks, 99.9% inside a gold execution, median relative position 0.13) and a confirm press closes it (1,752 presses, 1,751 of them inside a gold execution, exactly one per execution, position 0.89). Using both edges lifted boundary F1 to **0.756**, and at 2-second tolerance to **0.700** — against 0.316 and 0.240 for the best baseline.
3. **One screen pattern carries 38.3% of all observed work**, in all three portal systems. That is what the automation targeted first; reading each screen's printed table header, rather than assuming it, then carried the same engine to all 12 worklist screens.

The delivered tool covers **all 12 portal worklist screens — 984 rows, 93% fully automated, zero failures**. The 7% left for a person are precisely the rows the portal itself flags as needing judgment and for which no regulation rule resolves.

The recommendation I would defend hardest is a negative one: **the LLM does not belong in the runtime path**, and the evidence for that is in §5.

1. Step 1 — recovering units of work

What the logs do and do not contain

Process mining needs an event log of *(case ID, activity, timestamp)*. The provided data has timestamps only. Recovering the other two from an uncased stream is the known problem of *event log correlation*.

What did not work, and why I checked first

The obvious approach is to segment on pauses. I measured the gap distribution before building it: **the median gap between consecutive ground-truth segments is 0.0 seconds** — half of all true boundaries have no pause at all. Built anyway as a baseline, it reached BF1@5s 0.316 and V-measure 0.063. There is no threshold at which it works: at 2 s it emits 8,290 segments for 2,009 real ones; at 10 s it finds only 10% of true boundaries within 5 seconds.

A second baseline, splitting on every application switch, was also poor (V = 0.135): operators switch applications constantly *within* one unit of work.

The approach that did work

stage	signal	why
case identity	most-repeated ID in a screen capture	opening a record repeats its ID across header, fields and breadcrumb; list rows show each once. 93.5% precision
unit start	click on a table cell (`td`)	selecting the record; 1,780 clicks, 99.9% inside a gold execution, median position 0.13
unit end	click on an HTML `button`	the terminal action; 1,752 presses, 1,751 inside a gold segment (99.9%), exactly one per segment, position 0.89
label	portal system + route	3 systems x 5 routes is exactly the 15 process families; V = 0.931 on gold segments

Results

Scored against dataset A's 2,009 ground-truth executions:

	best baseline	delivered (v4)	ground truth
boundary F1 @2s	0.240	**0.700**	1.000
boundary F1 @5s	0.316	**0.756**	1.000
boundary F1 @10s	0.502	**0.818**	1.000
WindowDiff (lower better)	0.656	**0.195**	0.000
V-measure (label consistency)	0.135	**0.719**	1.000
segments produced	4,150	**2,010**	2,009
idle time claimed	41.5%	**6.6%**	5.0%

Two figures were never optimised for and are the ones I trust most: the segment count lands within **0.05%** of ground truth (2,010 against 2,009), and claimed idle time within 1.6 points.

How honest this number is

- **The scorer was validated first.** Scoring ground truth against itself returns 1.000 on every metric. Without that check the rest is unfalsifiable.
- **Two audits found real defects.** The boundary metric originally derived boundaries from label changes, which made the boundary between two consecutive executions of the *same* process invisible — it was scoring 94% of boundaries and silently discarding exactly the hardest 6%. A second pass found the fix had reached boundary F1 but not WindowDiff.
- **Parameter tuning leaked.** Parameters were swept over all 63 sessions with a dev/test split reported afterwards. Redone strictly, the honest protocol scored *higher* on test — the leak was real and was not inflating the result. Held out on the final pipeline: dev 0.748, **test 0.765**.
- **Generalisation was tested by holding out whole operators**, not random sessions: **BF1@5s 0.745, sd 0.119** across seven held-out machines. Holding out a whole department — the closer analogue of dataset B —

is not possible here: every one of the 63 sessions contains work from all three. The cross-department test is dataset B itself, checked against signals the pipeline never reads (§8).
- **The evaluator was itself audited for bias**, since I chose the tolerances, the binning and the gold construction. A random segmenter with the same segment count scores BF1@5s 0.267 and ARI 0.002 — the metric discriminates by **+0.489**. Shuffling labels while keeping boundaries collapses V-measure to 0.030, so the label score is not leaking from the segmentation. And a parameter-free check agrees independently: the best-matching predicted segment covers ≥80% of a ground-truth execution **83.0%** of the time, median coverage **97.5%**, against 17.8% and 38.1% for random.

Two things that audit changed. **BF1@5s has a floor near 0.26, not 0** — so the "best baseline" at 0.316 was barely above chance and is a weaker comparator than it appeared. And it surfaced a weakness the headline was understating: only 35.6% of ground-truth executions overlapped exactly one predicted segment. That was a real defect, and fixing it is what produced v4: the audit showed the ends were right and the starts approximate, so the opening click was brought in as the other bracket. **The figure is now 76.8%**, with median coverage 97.5% against a random control of 38%.

Where it fails, precisely

One held-out machine scores **0.471** against 0.72–0.86 for the other six. It is not a tuning artefact: **LAPTOP-R36BQBTE has zero L3 events across all seven of its sessions** — the browser extension never connected, so the route signal does not exist there. A fallback that recovers the route from the L2 accessibility layer limits the damage but does not remove it.

That gives the single most useful number in this report for planning purposes: **expect BF1@5s ≈ 0.75 ± 0.12 on an unseen operator, falling to ≈ 0.47 wherever L3 capture is missing.**

The residual limit is the 23.2% of executions that still do not map cleanly to a single segment. It is not where the opening click is missing - that describes one execution in 2,009. It splits between units with both clicks (61.4% of the misses) and the 258 units with neither, which the case-anchor fallback maps to a single segment only 30.2% of the time (`explore/error_sources.py`). Screen text is captured only every ~7.7 s, so case identity between observations is interpolated rather than seen.

Deliverable: `out/segments.jsonl` — 664 segments, 15 sessions, 12 labels.

---

2. Step 2 — what the work is

Scale and people

664 executions, 173 minutes, 15 sessions, 4 operators.

Three sources disagreed on headcount and had to be reconciled: 4 `username_hash` values, 4 machine IDs (strictly 1:1), but only 3 operator names on screen. The names are **not** operators — each appears on all four machines, and each maps to one portal system at 100% consistency. They are **shared per-system logins**. So: 4 people, working across three systems under shared accounts. That last part is a governance finding, not trivia (§6).

The route names are misleading

The portal is one SPA deployed three times, so route names repeat and describe nothing. The regulation document open during the work identifies it:

segment label	document consulted	what the work is
`ops__leave-applications`	keiyaku_kaijo_tetsuzuki	contract termination
`fin__leave-applications`	settai_keihi_kitei	entertainment expense approval
`fin__payroll-items`	getsujitsu_teigaku_torihikisaki_ichiran	recurring supplier payment
`fin__resident-tax`	shinkuitorihikisaki_touroku_tetsuzuki	new supplier registration
`hr__onboarding`	nyusha_checklist_shinsotsu_batch	new-graduate onboarding
`ops__social-insurance`	kanrisya_kengen_shinsei_tetsuzuki	admin privilege request

This doubles as **independent validation of Step 1's labels**: the pipeline never reads document names, yet segments sharing a label consult the same document at **80.4% weighted purity**.

(All Japanese readings in this report were verified by a Japanese-reading reviewer on 2026-09-10 — see `docs/CHECK_THIS.md`.)

Different handling within one process

The portal marks some rows for judgment: 種別 = 調整 (adjustment) on the three payroll-type screens, and 新規締結 or 解除 (a new contract, a termination) on the contract screen. Through a case ID seen inside the segment, **578 of the 664 segments (87.0%)** can be tied to the worklist row they processed, so flagged and routine rows can be compared within each process:

process	routine runs	flagged runs	median, routine → flagged	keystrokes per run	regulation open
payroll_item_maintenance	93	20	11.0 → 11.5 s	4.3 → 5.3	13% → 25%
inventory_payroll_items	18	43	11.0 → 11.0 s	4.1 → 10.3	0% → 5%
recurring_supplier_payment	50	19	17.0 → 16.0 s	11.7 → 11.8	72% → 63%
contract_termination	12	30	16.0 → 20.0 s	5.5 → 9.7	92% → 87%

No difference survives a correction for the 12 comparisons made (permutation tests, Bonferroni; the smallest p is 0.08, for flagged contract_termination rows taking 4.0 s longer). On everything the logs record, flagged rows are handled much like routine ones: whatever judgment they need leaves no trace in time, typing or regulation reading - in a test environment whose waiting was compressed. So the portal's flag, not a measured difference in effort, is the evidence for leaving those rows to a person, and the time automation saves does not hinge on which rows are flagged. With 19–43 flagged runs per process only a large difference could have shown (`explore/handling_variants.py`).

Ranking, and why it is ranked this way

Two measurable quantities in tension:

- **Mechanical load** — clipboard events and application switches per run. What automation removes.
- **Judgment load** — share of runs with a regulation document open. What it does not.

High volume with high judgment is a poor first target however expensive it looks, because the residue is what costs the time.

process	n	min	share	median	mech	judgment
payroll_item_maintenance	120	27.2	15.8%	11 s	2.6	15%
recurring_supplier_payment	77	23.4	13.6%	16 s	3.9	65%
new_grad_onboarding	58	19.2	11.1%	19 s	6.2	88%
contract_termination	59	19.1	11.1%	19 s	6.2	88%
new_supplier_registration	73	15.6	9.0%	11 s	2.1	27%
inventory_payroll_items	63	15.5	9.0%	11 s	3.1	3%
leave_application_review	58	13.2	7.7%	10 s	2.0	3%
admin_privilege_request	42	8.5	4.9%	11 s	2.7	17%

A priority formula is easy to make say what you want, so the ranking was scored under **six different weightings**. Only **payroll_item_maintenance** appears in the top three under all six. Three processes appear exactly once, which makes them artefacts of a particular formula rather than findings.

The result that set the scope

Aggregating by portal **screen** instead of by system. Judgment is the share of the screen's runs with a regulation document open:

screen	executions	minutes	share	systems	judgment
payroll-items	**260**	**66.1**	**38.3%**	**3**	**27%**
leave-applications	151	41.8	24.2%	3	54%
onboarding	95	30.1	17.5%	2	85%
social-insurance	85	18.9	11.0%	3	28%
resident-tax	73	15.6	9.0%	1	27%

The top-ranked processes are the same screen in different deployments. One pattern, 38% of all work, and the lowest judgment load of the five screens — though only just, level with resident-tax and social-insurance at 27–28%. That is the target.

---

3. Step 3 — what I built

A **shared worklist automation engine** driven by one definition file per screen, covering all 12 worklist screens across the three systems.

```
 tool/engine.py              the automation logic - one copy
tool/definitions/*.yaml     what differs per screen: URL, columns, regulation, wording
tool/regulations.py         extracts decision rules from the 規程 text
tool/mock_portal/           the portal, reconstructed from the logs
```

Measured result — all 984 rows across 12 screens

outcome	rows	share
routine, templated note	821	83%
flagged rows routed by regulation threshold	94	10%
fully automated	**915**	**93%**
left for a person	69	7%
failed	**0**	**0%**

Median 133 ms per row, p95 186 ms, 166 s for the full set.

The scope grew after a defect was found. The first build modelled 3 screens and 456 rows, because the fixture extractor kept only the first breadcrumb per system and hard-coded the payroll column list — so it silently found only the screens whose table matched. Reading the printed header instead of assuming it exposed **12 screens, 984 rows and 5 distinct schema archetypes**. The earlier claim that the three systems share an identical table contract was true across systems for one screen, and false across screens.

Why this process and this scope

Why this process: it is the only candidate that survived all six ranking weightings, and its screen pattern accounts for 38.3% of observed work.

Why this scope: the brief notes that breadth-versus-depth is itself the ROI question. The first build targeted this one screen, on the evidence that all three systems render it with an *identical table contract* — same seven columns, same element ids, differing only in content (`E2001` vs `BATCH-W2` , yen vs an em dash). That held across systems and not across screens, but the engine turned out not to need it: it reads each screen's printed header, so the five schema archetypes are data rather than code. One engine plus 12 definition files, all but one under 55 lines, covers every portal worklist screen; twelve bespoke scripts would repeat the same control flow twelve times.

What I deferred: the 10 of 13 regulation documents that carry no machine-readable thresholds. They are procedural checklists, and handling them means solving procedure-following rather than rule lookup — a different and harder problem. Rows governed by those still reach a person.

---

4. Why this implementation form, and what I rejected

option	why not
RPA tool (UiPath, Power Automate)	Would work. Rejected on operational grounds: the client already captures DOM selectors, so a code path is more testable, diffable and reviewable than a recorded flow — and the per-system difference is data, which suits a config file rather than three recorded macros.
An LLM agent driving the UI	Measured and rejected. See below.
API integration	Preferable if it exists, but nothing in the logs evidences an API. Proposing one would be an assumption, and the brief warns against exactly that. Flagged as the first thing to check in a real engagement (§6).
A script per screen	Rejected: the 12 screens share five table schemas and one control flow, so twelve scripts would repeat that flow twelve times.
Deterministic engine + config	**Chosen.**

The LLM decision, in detail

This was designed as a RAG feature: read the 規程, decide the handling. Two pieces of evidence removed it.

Measured performance. Running the judgment step through Gemini:

	deterministic	model
median latency per row	170 ms	23,310 ms (**135x**)
errors	0 / 456 (the rows then modelled)	2 / 5 (timeouts)
output quality	states the facts	echoed the regulation's *filename* back

And then the reason it was never needed. The captured regulation text is a threshold table, not a judgment:

> 第２条（承認権限）1回あたり5万円未満：部門長承認。5万円以上：役員承認。10万円以上：社長承認。
>
> *Article 2 (Approval authority). Under ¥50,000 → department head; ¥50,000 and over → executive; ¥100,000 and over → president.*

Approval routing by amount is **arithmetic**. Arithmetic belongs in code: exact, instant, free, auditable line by line against the regulation, and incapable of inventing an approver. `regulations.py` parses those thresholds; `route(107158)` returns 社長承認; the tool writes `規程により社長承認へ回付`.

The model keeps at most one job, and it would run offline: proposing a rule table from a regulation document for a human to check before it ships. The expensive, unreliable, unauditable component would run *once under supervision* rather than on every transaction. That job is designed, not built: all three regulations that carry thresholds were parsed without a model. In the tool the model is off by default, even with a key configured; `--llm-drafts` switches on only the drafting experiment measured above.

I would defend this as the correct answer rather than a compromise. An LLM in this path would have been slower, less reliable, more expensive, unauditable against the regulation it is supposed to implement — and would have

looked more impressive.

---

5. What manual work remains, and realistic impact

Remains, by construction

- **69 of 984 rows (7%)** — every row the portal flags for confirmation that no regulation threshold resolves: 43 contract rows (31 new agreements, 12 terminations) and 26 inventory adjustments whose 金額 is an em dash, where with no amount the threshold table says nothing. Correct boundary, not a gap.
- **The other 10 of 13 regulation documents** carry no machine-readable thresholds; they are procedural checklists. Processes governed by those are out of scope entirely.
- **Exception handling.** Nothing in the logs shows what operators do when a record is malformed or a system rejects a submission, so the tool has no designed behaviour for it beyond failing loudly.
- **Review of the automated output.** At least during rollout, a person should sample what the tool wrote.

Impact, stated carefully

The brief states the recordings were made in a test environment with compressed waiting times, so **absolute durations here cannot be converted into hours saved, and I will not do so.** What the evidence supports is relative:

- Every segment in `segments.jsonl` is portal work, and the tool now covers **all 12 portal worklist screens**.
- Across them, **93% of rows** were handled end to end without a person.
- So the defensible claim is that the tool addresses **the portal component of the observed workload, at 93% coverage within it** — under favourable conditions (§6).

What it does *not* support: any statement of hours or money saved. Producing one would require production timings the client has and I do not.

---

6. Risks, and the evidence for each

Every risk below is anchored to a measurement rather than to a worry.

#	risk	evidence	mitigation
R1	**Telemetry gaps silently degrade accuracy.**	One of seven held-out machines scored BF1@5s **0.471** vs 0.72–0.86. Cause: **zero L3 events across all 7 of its sessions** — the extension never connected. Dataset B has the same gap in **1 of its 15 sessions**: 22 of its 664 segments, 6.3% of the time, come from the weaker fallback.	Monitor L3 coverage per machine as a first-class health metric; refuse to report process figures for a machine below a coverage floor. The L2 accessibility fallback limits but does not remove the damage.
R2	**The mock portal is not the real portal.**	Session handling, server-side validation, pagination, concurrency and real latency are **unobserved in the logs** and therefore unimplemented.	Treat 93% as an upper bound under favourable conditions. First engagement task: run against a staging instance before any efficiency claim is repeated.
R3	**An approach validated on one department can fail silently on another.**	Three transfers failed during this project: the case-ID prefix (100% pure on A, meaningless on B), the anchor rule (fired 72 times in all of B), and the button naming (`btn-*-ok` matched **zero** rows in A). Each looked fine on internal statistics.	Never accept a transfer on internal statistics alone. Hold back one observable signal from the method and check against it — that is exactly what caught the first dataset B failure.
R4	**Automation runs under a shared account.**	Each portal system has one login shared by all four operators (100% name-to-system consistency).	No per-user audit trail exists today, so automated and human actions will be indistinguishable in the client's own logs. Needs a service account with a distinct identity before rollout, which is an access-control change, not a code change.
R5	**The regulation is the specification, and it changes.**	Thresholds are hard rules parsed from document text (`5万円未満：部門長承認`). A revised 規程 silently invalidates them.	Rules are extracted from the document rather than typed into code, so re-extraction is the update path. Version the rule table against the document; alert on drift; require human sign-off on each extraction.
R6	**Provider availability, if a model is ever added.**	The first live API call fell through **two 503s** before succeeding, and the default model had been retired for new keys mid-project.	Already mitigated by design: the model is off the runtime path, and the tool works with none configured.
R7	**Free-tier LLM data is used for provider training.**	Vendor terms.	No model runs unless explicitly enabled. When one is, `src/llm.py` refuses — on the only path that sends data — any prompt carrying a raw event record, a session id or a case or row id, or over 20,000 characters: enforced in code. It cannot recognise every kind of personal text, such as a name, so the one prompt the tool can send is built from a row's type, amount and classification — never its id or names.
R8	**Boundary precision is structurally limited.**	BF1@2s = 0.700 against 0.818 at 10 s — about 30% of true boundaries are not found within 2 s; screen text is captured every ~7.7 s.	Do not build anything requiring sub-5-second boundary accuracy. If needed, raise capture frequency — a collection change, not an algorithm change.

---

7. How the time was spent

A disclosure first: this was not seven calendar days. The git history runs from 9 to 11 September — three days, not a week. The brief asks how the seven days were allocated, and the honest answer is that the work was compressed. What follows is how the *effort* divided, which is the part that carries a lesson.

phase	work	why
1	Data profiling; 790 MB of JSONL → an 11 MB index	Nothing else was affordable to iterate on until this existed
2	**Evaluation harness before the segmenter**; baselines	The brief leaves "good enough" to my judgment, and that judgment is impossible without a scorer. It also killed the gap-based approach in an hour rather than a day
3	Case-ID recovery	Reframed the problem, though the final architecture barely uses it (see below)
4	v1 segmenter; LLM layer	BF1@5s 0.450. The LLM layer was premature
5	Signature labelling; **dataset B transfer failed**; two metric audits	The failure was the most valuable hour of the project
6	Terminator-driven segmenter; overfitting audit; Step 2	BF1@5s 0.450 → 0.722
7	Step 3 engine, three definitions, mock portal	94% automated, zero failures
8	Report; **evaluator bias audit**, which exposed a defect and produced v4	BF1@2s 0.286 → **0.673**
9	Continued defect hunting, as a reviewer would	Optimal boundary matching rescored everything, baselines included (BF1@2s **0.700**); Step 3 generalised from 3 screens to all 12 (93% of 984 rows); drift detection; asserted tests, each checked to fail

The allocation I would defend: roughly a third of the effort went to measurement rather than building — the harness, four audits, the held-out protocol, the transfer checks. That is a high proportion and it paid for itself repeatedly. The metric defect would have invalidated every subsequent number. The dataset B transfer failure was invisible on every internal statistic. The evaluator audit, run only because the approach was challenged, produced the single largest accuracy gain in the project.

What the final ablation says about that allocation

Removing each component and re-scoring (`explore/ablation.py`):

configuration	BF1@2s	V	ARI
shipped	0.700	0.719	0.708
without case anchors entirely	**0.723**	0.694	0.585
ends only, no opening click	0.316	0.562	0.540
labels from case prefix	0.700	0.471	0.385
one label for everything	0.700	0.011	−0.001

Two uncomfortable readings, both worth stating.

Case-ID recovery contributes nothing to boundary accuracy. Boundaries are marginally *better* without it at 2 s (0.723 against 0.700) and marginally worse at 5 s (0.750 against 0.756). It earns its place only by driving the fallback that recovers the 260 segments no confirm press brackets — which shows in ARI (0.708 vs 0.585) — and by supplying the case field used to validate dataset B. A day of work that the final architecture largely routed around.

The whole boundary gain is two clicks. Everything else — the anchors, the snapping, the tuned windows — is close to inert. That is the honest description of where the accuracy comes from, and it is also why the result is robust: `expand_gap_s` from 20 to 300 and `max_unit_s` from 100 to 600 leave BF1@2s unchanged at 0.700; `min_unit_s` from 1 to 10 moves it by at most 0.003, and the case-anchor threshold from 2 to 4 spans 0.695–0.714. That threshold was chosen for anchor precision, not for this score, and moving it to 4 now because it scores higher here would be tuning on the evaluation set. **There is almost nothing here that could be overfitted, because almost nothing is fitted.**

Three further attempts to raise accuracy were measured, and none shipped. Each was judged against a bar written down before its full evaluation. The strongest divided the gap between units at its first app switch: boundaries improved, but label consistency fell beyond the tolerance set in advance, and one machine got worse. The delivered pipeline is unchanged, and the attempts and their numbers are in `RESULTS.md`.

The phase I would remove: the LLM layer, built well before anything needed it, on an assumption that the task wanted one. The eventual conclusion was that it should not be in the runtime path at all. That effort belonged in Step 2.

8. Honest limitations

- **Dataset B has no ground truth**, so its accuracy is not measured, only inferred from dataset A performance and held-out proxy checks. The one that still discriminates: labels agree with the portal breadcrumb — an L1 signal the labeller cannot see — at **V = 0.948** (96.8% on the system) where a breadcrumb was captured within 2 s of the segment's end, on 31 segments, against 0.158 for shuffled labels. Against the breadcrumb captured anywhere in the segment it falls to 0.480, and against the one most often in force to 0.255, because screen text is captured only every ~7.7 s and most references are stale by the time a segment ends (`explore/label_vs_breadcrumb.py`).
- **The completion-memo check no longer discriminates.** All 149 memos, which the pipeline never reads, fall inside a segment at median relative position 0.47 — but segments now cover 97.9% of session time, and 5,000 random instants give 97.9% and 0.50. Under v3, whose segments ended at the confirm press, the memos sat at 0.78; v4 extends every segment past that press. The check is reported as spent rather than quoted as evidence.
- **No measurable difference between flagged and routine rows is not proof of none.** 578 of the 664 segments tie to their worklist row (§2), but each process has only 19–43 flagged runs, so the comparison can see a large difference and not a small one.
- **10% of dataset A screenshots are missing** from the distribution provided. Not pursued: dataset B is complete and screen text is available as text.
- **"The portal component at 93% coverage" describes 173 observed minutes on one day with four operators.** It is a measurement of this sample, not an estimate of the client's operation.

What I would do next, in order

1. Run the engine against a staging instance of the real portal (R2).
2. Check whether an API exists behind the portal. If it does, most of this tool becomes unnecessary, and that is a good outcome.
3. Instrument L3 coverage per machine as a health metric (R1).
4. Take on the 69 rows still left for a person — contract rows and inventory adjustments with no amount. That is the procedure-following problem deferred in §3, not a matter of adding screens.